

Árboles (CARTs)

Big Data y Machine Learning para Economía Aplicada

Ignacio Sarmiento-Barbieri

Universidad de los Andes

Agenda

1 Recap

2 Árboles

- Árboles de Regresión
- Árboles de Clasificación
- Sobreajuste

Recap

- Queremos predecir y en función de observables ($\mathbf{x}_i$)

$$y = f(\mathbf{x}_i) + u \quad (1)$$

- donde la estimación de f implica la que minimize el riesgo empírico (prediga mejor fuera de muestra):

$$\hat{f} = \underset{f}{\operatorname{argmin}} \left\{ \sum_{i=1}^n L(y_i, f(\mathbf{x}_i; \Theta)) \right\} \quad (2)$$

Recap

- Por ejemplo en regresión $f(x_i) = \mathbf{x}_i\beta$ y minimizamos

$$\hat{\beta} = \underset{\tilde{\beta}}{\operatorname{argmin}} \left\{ \sum_{i=1}^n (y_i - \mathbf{x}_i\beta)^2 \right\} \quad (3)$$

- Por ejemplo en clasificación logística $p(x_i) = \frac{1}{1+e^{-\mathbf{x}_i\beta}}$

$$\hat{\beta}^{MLE} = \underset{\beta}{\operatorname{argmin}} - \left[\sum_{i=1}^n \log \left(\frac{p_i}{1-p_i} \right)^{y_i} + \sum_{i=1}^n \log(1-p_i) \right] \quad (4)$$

Agenda

1 Recap

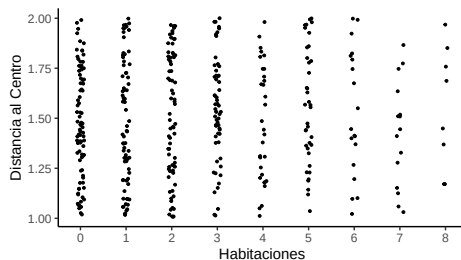
2 Árboles

- Árboles de Regresión
- Árboles de Clasificación
- Sobreajuste

Motivación

- Queremos predecir:

$$\text{Precio} = f(\text{habitaciones}, \text{Distancia al CBD}) \quad (5)$$



Motivación

- Podemos asumir f es lineal

$$f(\mathbf{x}_i; \beta) = \beta_0 + \beta_1 \text{Habitaciones}_i + \beta_2 \text{DCBD}_i + u_i \quad (6)$$

Motivación

► o inspirarnos por Leo Breiman:

"There are two cultures in the use of statistical modeling to reach conclusions from data. One assumes that the data are generated by a given stochastic data model. The other uses algorithmic models and treats the data mechanism as unknown." Breiman [2001b], p199.

"The statistical community has been committed to the almost exclusive use of data models. This commitment has led to irrelevant theory, questionable conclusions, and has kept statisticians from working on a large range of interesting current problems. Algorithmic modeling, both in theory and practice, has developed rapidly in fields outside statistics. It can be used both on large complex data sets and as a more accurate and informative alternative to data modeling on smaller data sets. If our goal as a field is to use data to solve problems, then we need to move away from exclusive dependence on data models and adopt a more diverse set of tools." Breiman [2001b], p199.

Motivación

- Podemos asumir f es lineal

$$f(\mathbf{x}_i; \beta) = \beta_0 + \beta_1 \text{Habitaciones}_i + \beta_2 \text{DCBD}_i + u_i \quad (6)$$

- o inspirarnos por Leo Breiman y ajustar un modelo flexible e interpretable como son los arboles

$$f(x_i; \{R_j, \gamma_j\}_1^J) = T(x_i; \{R_j, \gamma_j\}_1^J) \quad (7)$$

Agenda

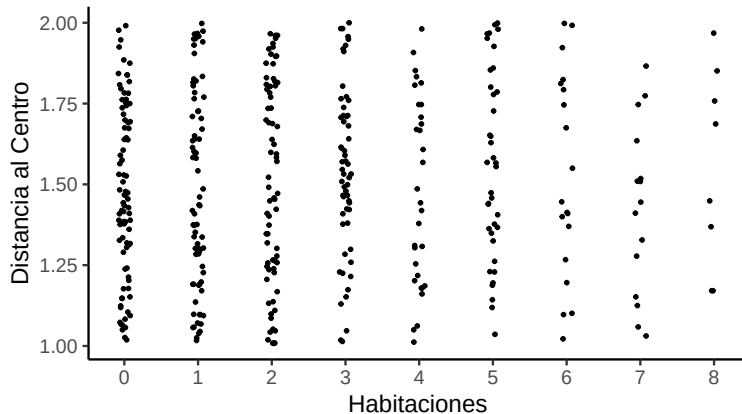
1 Recap

2 Árboles

- Árboles de Regresión
- Árboles de Clasificación
- Sobreajuste

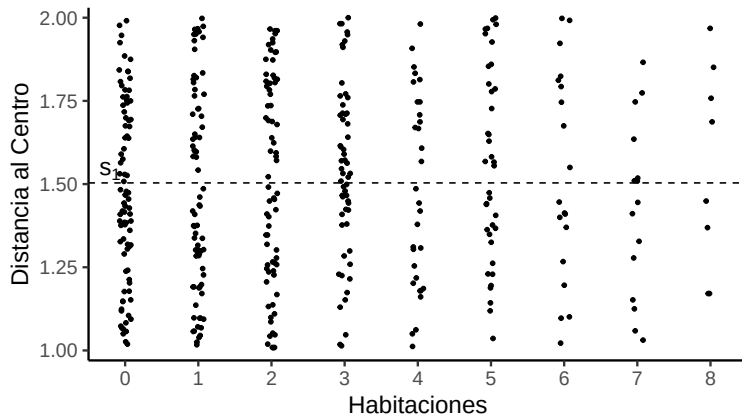
¿Qué hacen?

“Recursive binary splitting”



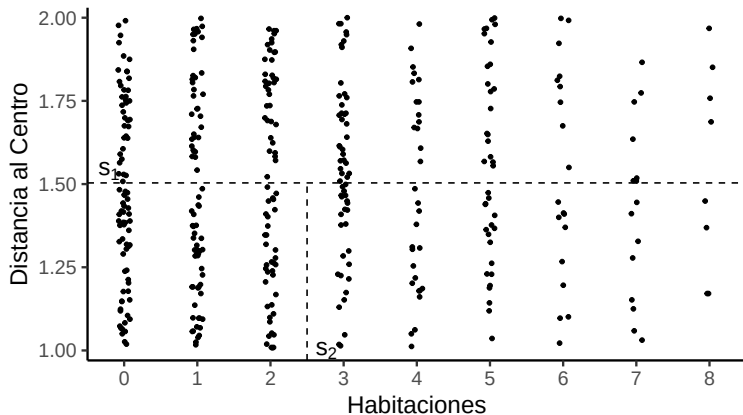
¿Qué hacen?

“Recursive binary splitting”



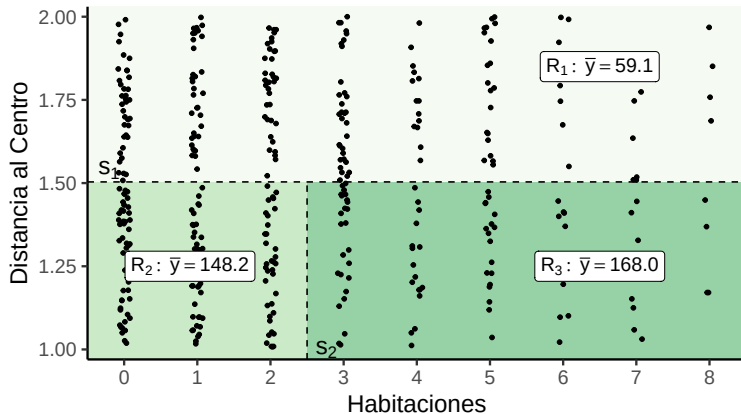
¿Qué hacen?

“Recursive binary splitting”



¿Qué hacen?

“Recursive binary splitting”



En cada región R_m la predicción es la media: $\hat{y} = \bar{y}_{R_m}$.

¿Cómo construimos un árbol?

- El problema de optimización

$$\hat{f} = \operatorname{argmin}_f \left\{ \sum_{i=1}^n L(y_i, f(\mathbf{x}_i; \Theta)) \right\} \quad (8)$$

- es ahora

$$\{\hat{R}_j, \hat{\gamma}_j\}_1^J = \operatorname{argmin}_{\{R_j, \gamma_j\}_1^J} \left\{ \sum_{i=1}^n L(y_i, T(\mathbf{x}_i; \{R_j, \gamma_j\}_1^J)) \right\} \quad (9)$$

¿Cómo construimos un árbol?

- ▶ Datos: $y_{n \times 1}$ y $X_{n \times k}$
- ▶ Definiciones
 - ▶ j es la variable que parte el espacio y s es el punto de partición
 - ▶ Defina los siguientes semiplanos

$$R_1(j, s) = \{X | X_j \leq s\} \ \& \ R_2(j, s) = \{X | X_j > s\} \quad (10)$$

- ▶ El problema: usando una “perdida cuadrática” buscar la variable de partición X_j y el punto s de forma tal que:

$$\min_{j,s} \left[\min_{\gamma_{R_1}} \sum_{x_i \in R_1(j,s)} (y - \gamma_{R_1})^2 + \min_{\gamma_{R_2}} \sum_{x_i \in R_2(j,s)} (y - \gamma_{R_2})^2 \right] \quad (11)$$

¿Cómo construimos un árbol?

- ▶ ¿Cuál es la solución?

¿Cómo construimos un árbol?



photo from <https://www.dailydot.com/parsec/batman-1966-labels-tumblr-twitter-vine/>

Agenda

1 Recap

2 Árboles

- Árboles de Regresión
- Árboles de Clasificación
- Sobreajuste

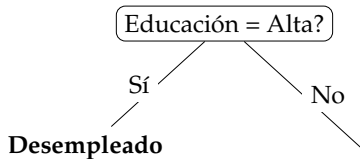
Árboles: Problema

- ¿Está desempleado?

Educación	Edad	Desempleado?
Baja	22	Sí
Baja	25	Sí
Alta	28	No
Baja	30	Sí
Baja	35	No
Alta	40	No
Alta	38	No

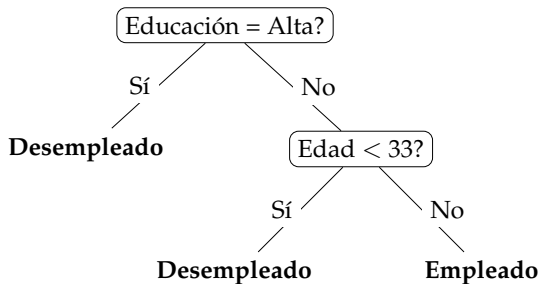
Árboles: Problema

Educación	Edad	Desempleado?
Baja	22	Sí
Baja	25	Sí
Alta	28	No
Baja	30	Sí
Baja	35	No
Alta	40	No
Alta	38	No



Árboles: Problema

Educación	Edad	Desempleado?
Baja	22	Sí
Baja	25	Sí
Alta	28	No
Baja	30	Sí
Baja	35	No
Alta	40	No
Alta	38	No



¿Cómo construimos un árbol de decisión?

- ▶ Regiones lo más “puras” posibles
 - ▶ **Regresión:** mínima varianza
 - ▶ **Clasificación:** ?

¿Cómo construimos un árbol de decisión?

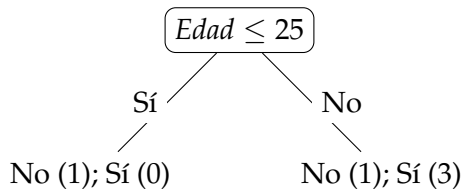
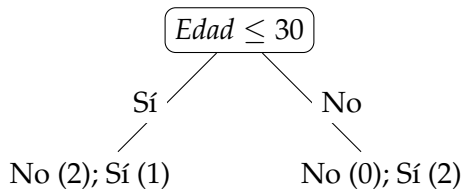
Problemas de clasificación

Edad	Desempleado?
22	No
25	Sí
30	No
35	Sí
40	Sí

¿Cómo construimos un árbol de decisión?

Problemas de clasificación

- ¿Cuál de los dos cortes es mejor?



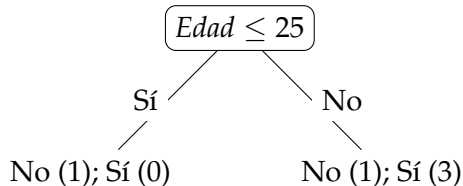
¿Cómo construimos un árbol de decisión?

Problemas de clasificación. Medidas de Impureza

- ▶ Medidas de impureza dentro de cada hoja:
 - ▶ Índice de Gini : $G = \sum_{k=1}^K \hat{p}_{mk}(1 - \hat{p}_{mk})$
 - ▶ Entropía : $H = - \sum_{k=1}^K \hat{p}_{mk} \log(\hat{p}_{mk})$
- ▶ Se define la impureza de un árbol por el promedio ponderado de las impurezas de cada hoja. El ponderador es la fracción de observaciones en cada hoja.

¿Cómo construimos un árbol de decisión?

Problemas de clasificación. Impureza — Corte B: $Edad \leq 25$



Hoja izq. ($n_L = 1$): No(1), Sí(0) — *pura*

$$G_L = 0$$

Hoja der. ($n_R = 4$): No(1), Sí(3)

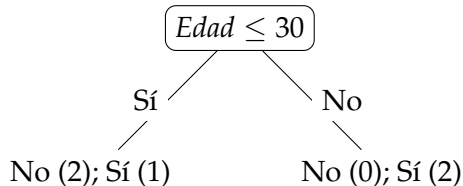
$$G_R = 2 \cdot \frac{1}{4} \cdot \frac{3}{4} = \frac{3}{8} = 0.375$$

Impureza ponderada:

$$G_B = \frac{1}{5} \cdot 0 + \frac{4}{5} \cdot \frac{3}{8} = \frac{3}{10} = \mathbf{0.30}$$

¿Cómo construimos un árbol de decisión?

Problemas de clasificación. Impureza — Corte A: $Edad \leq 30$



Hoja izq. ($n_L = 3$): No(2), Sí(1)

$$G_L = 2 \cdot \frac{2}{3} \cdot \frac{1}{3} = \frac{4}{9} \approx 0.44$$

Hoja der. ($n_R = 2$): No(0), Sí(2) — *pura*

$$G_R = 0$$

Impureza ponderada:

$$G_A = \frac{3}{5} \cdot \frac{4}{9} + \frac{2}{5} \cdot 0 = \frac{4}{15} \approx \mathbf{0.27}$$

¿Cómo construimos un árbol de decisión?

Problemas de clasificación. Comparación

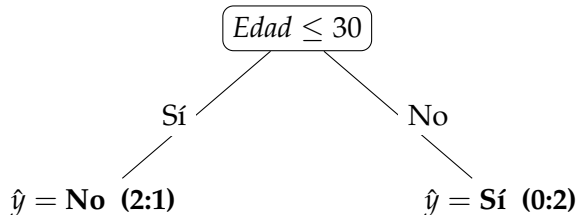
- ▶ Ambas medidas eligen el mismo corte óptimo:

Corte	Gini	Entropía
$Edad \leq 30$	0.27	0.55
$Edad \leq 25$	0.30	0.65

¿Cómo construimos un árbol de decisión?

Problemas de clasificación. Predicción

► Predicción



Medidas de Impureza: Gini vs. Entropía

Índice de Gini

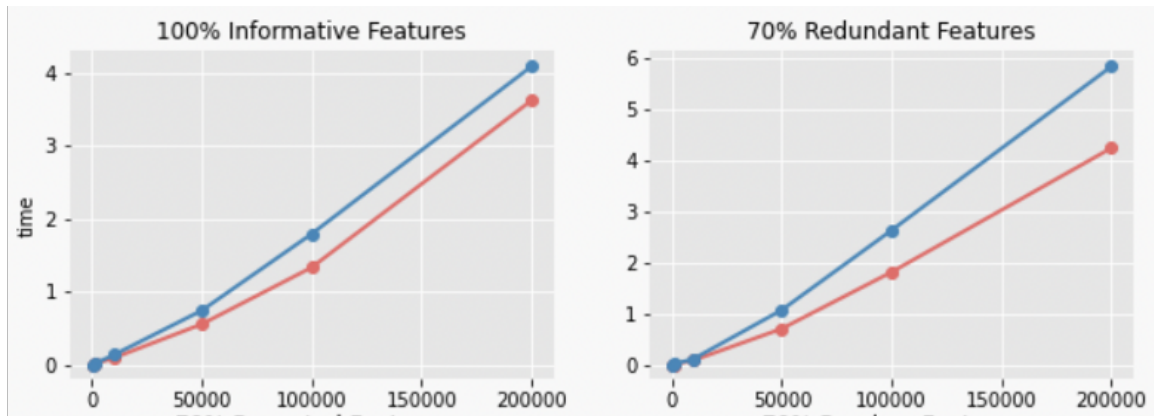
$$G = 1 - \sum_k p_k^2 = \sum_k p_k(1 - p_k)$$

Entropía (cross-entropy)

$$H = - \sum_k p_k \log_2 p_k$$

- ▶ Mínimo = 0 en nodos puros ($p_k = 1$)
- ▶ Máximo en distribución uniforme
- ▶ Ambas son cóncavas
- ▶ Ambas producen resultados similares en la práctica

Medidas de Impureza: Gini vs. Entropía



Gini vs. Entropía: ¿Cuál usar?

Gini (CART clásico)

- ▶ Más rápido (evita logaritmos)
- ▶ Menos sensible a clases raras
- ▶ Tiende a aislar la clase más frecuente en una rama
- ▶ Default en R

Entropía

- ▶ Más sensible a pureza extrema
 - ▶ Penaliza más la presencia de clases raras
 - ▶ Puede favorecer splits que aíslan clases minoritarias
 - ▶ Útil cuando la clase rara importa
-
- ▶ En la práctica, la diferencia en rendimiento suele ser pequeña.
 - ▶ Con clases desbalanceadas, a menudo es más importante ajustar los pesos de las clases o el umbral de decisión que cambiar la métrica de impureza.

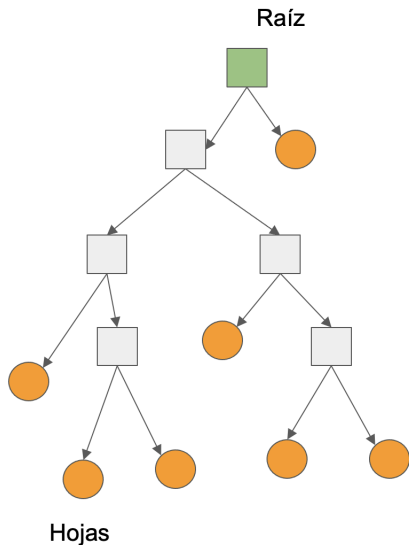
Agenda

1 Recap

2 Árboles

- Árboles de Regresión
- Árboles de Clasificación
- **Sobreaajuste**

Sobreajuste



Sobreajuste. Algunas soluciones

- ▶ Fijar la profundidad del árbol.
- ▶ Fijar la mínima cantidad de datos que están contenidos dentro de cada hoja.
- ▶ Pruning (poda).
 - ▶ Dejar crecer un árbol muy grande T_0
 - ▶ Luego cortarlo obteniendo sub-árbol (*subtree*)
 - ▶ Como cortarlo?

Pruning (poda)

- ▶ No es posible calcular el error de predicción usando cross-validation para cada sub-árbol posible
- ▶ Solución: *Cost complexity pruning* (cortar las ramas mas débiles)
 - ▶ Indexamos los arboles con T .
 - ▶ Un sub-árbol $T \in T_0$ es un árbol que se obtuvo colapsando los nodos terminales de otro árbol (cortando ramas).
 - ▶ $[T]$ = número de nodos terminales del árbol T

Pruning (poda)

- Cost complexity del árbol T

$$C_\alpha(T) = \sum_{m=1}^{[T]} n_m Q_m(T) + \alpha [T] \quad (12)$$

- donde $Q_m(T) = \frac{1}{n_m} \sum_{x_i \in R_m} (y_i - \hat{y}_m)^2$ para los árboles de regresión
- $Q_m(T)$ penaliza la heterogeneidad dentro de la regresión y α el número de regiones
- Objetivo: para un dado α , encontrar el pruning óptimo que minimice $C_\alpha(T)$

Pruning (poda)

- Mecanismo de búsqueda para T_α (pruning óptimo dado α).

Resultado: para cada α hay un sub-árbol único T_α que minimiza $C_\alpha(T)$.

- Eliminar sucesivamente las ramas que producen un aumento mínimo en $\sum_{m=1}^{[T]} n_m Q_m(T)$
- Se colapsa hasta el nodo inicial pero va a través de una sucesión de árboles
- T_α pertenece a esta secuencia. (Breiman et al., 1984)

Pruning (poda)

Algoritmo Completo

- 1 Utilizamos particiones recursivas binarias para hacer crecer el árbol
- 2 Para un dado α , aplicamos *cost complexity pruning* al árbol para obtener la secuencia de los subárboles como α .
- 3 Utilizamos K-fold cross-validation para elegir α .
- 4 Tenemos entonces una secuencia de subárboles para distintos valores de α
- 5 Elegimos el α y el subárbol que tienen el menor error de predicción.

Pruning (poda) para Clasificación

- ▶ La misma idea aplica para árboles de clasificación: dejar crecer T_0 y luego podar.
- ▶ La medida de impureza $Q_m(T)$ se reemplaza por Gini o Entropía:

$$C_\alpha(T) = \sum_{m=1}^{[T]} n_m Q_m(T) + \alpha [T] \quad (13)$$

donde

$$Q_m(T) = \sum_k \hat{p}_{mk}(1 - \hat{p}_{mk}) \quad (\text{Gini})$$

- ▶ $n_m Q_m(T)$: impureza total de la hoja m .
- ▶ $\alpha [T]$: penalización por complejidad del árbol ($[T]$ = número de hojas).
- ▶ Un α grande favorece árboles simples; $\alpha = 0$ recupera T_0 .

Ejemplo



photo from <https://www.dailydot.com/parsec/batman-1966-labels-tumblr-twitter-vine/>